RDT-UDP

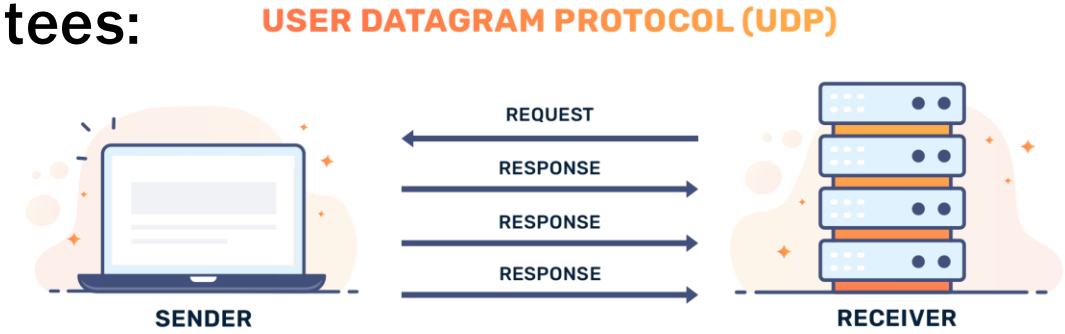
By: Joe Jimenez and Alan Lam

CS5470: Computer Networks

THE PROBLEM

UDP delivers datagrams with no guarantees:

- packets can be lost
- packets can be corrupted
- packets can arrive out of order
- packets can be duplicated



Many applications still need reliable, ordered delivery

TCP provides this, but hides the mechanisms behind it

Our task: rebuild those guarantees on top of UDP

WHAT WE BUILT

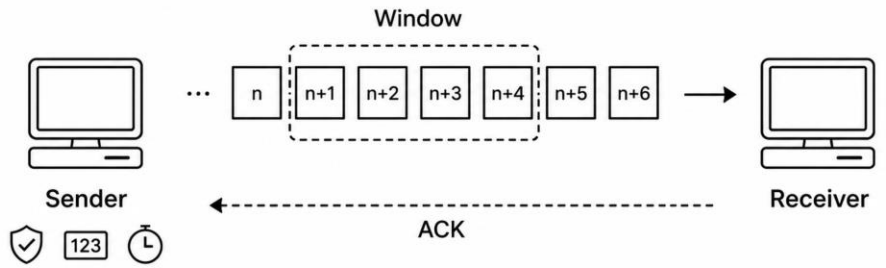
Sender - splits messages, assigns sequence numbers and checksums, sends within a configurable window, retransmits on timeout

Receiver - validates checksums and ACKs received packets

Reliability mechanisms:

- Checksums - detect corruption
- Sequence numbers - detect loss & order packets
- ACKs - confirm delivery
- Timers + retransmission - recover from loss
- Sliding window - multiple packets in flight

Configurable: host, port, timeout, payload size, window size, max retries



PACKET FORMAT

Packet Format

seq_num | ack_num | payload | checksum

Fields

- seq_num = packet sequence number
- ack_num = sequence number being acknowledged
- payload = message data (empty for ACKs)
- checksum = 8-bit checksum to detect corruption

Example Data Packet

- 3 | 0 | hello world | 1463 → Sequence 3, message "hello world"

Example ACK Packet

- 0 | 3 | | 347 → Acknowledging sequence 3

Text format is easy to debug, log, and parse during testing.

CHECKSUM

- Cyclic redundancy check (CRC) is an error-detection technique
- Data is encoded as a binary string (d bits)
- CRC algorithm takes in d bits as input and outputs r bits, which we use as the checksum value

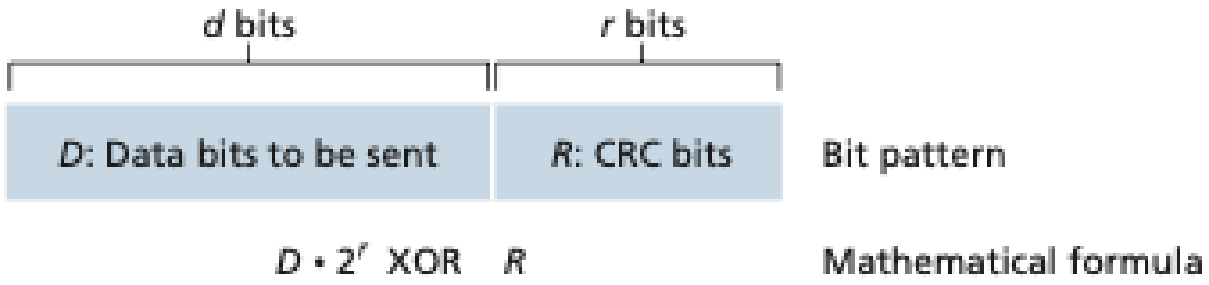


Figure 6.6 ♦ CRC

CHECKSUM

- Data string D is divided by a pre-determined generator string G
- The result is the remainder R
- Idea: if 2 binary strings have the same remainder when divided by G, then they are likely the same string
  - G should be fairly large so that the total number of possible remainders is large

```
def divide_modulo_2(dividend, divisor):
    dividend_index = len(divisor)
    dividend_substr = dividend[0 : dividend_index]

    while dividend_index < len(dividend):
        # Divisor "divides" into the dividend substring
        # If dividend substring starts with a 1, divisor and dividend are the same length
        if dividend_substr[0] == '1':
            dividend_substr = xor(dividend_substr, divisor) + dividend[dividend_index]

        # Divisor "doesn't divide" into the dividend substring
        # If dividend substring starts with a 0, divisor > dividend_substr
        else:
            dividend_substr  = xor(dividend_substr, '0' * dividend_index) + dividend[dividend_index]
```

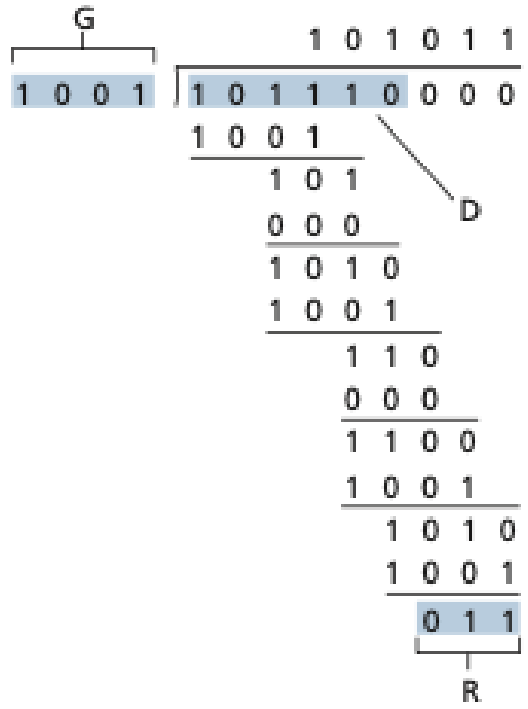


Figure 6.7 ♦ A sample CRC calculation

SENDER LOGIC

- Splits the message into fixed-size payload chunks
- Assigns each packet a sequence number and checksum
- Sends up to window_size packets at a time
- Tracks unacknowledged packets in a sender window
- Removes packets from the window when valid ACKs arrive
- Retransmits packets whose timer expires before an ACK arrives

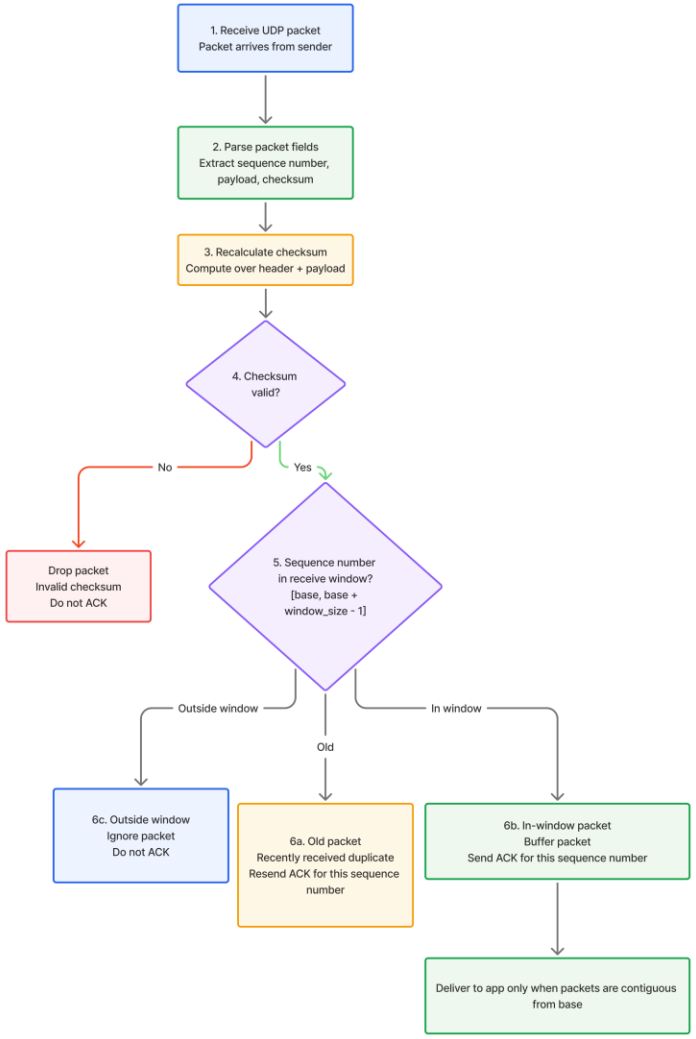
SENDER: RETRANSMIT & ACK VALIDATION

```
def _retransmit_timed_out_packets(self, states, window_base, window_limit):
    now = time.monotonic()
    for seq_num in sorted(states):
        if seq_num < window_base or seq_num >= window_limit:
            continue
        last_sent = states[seq_num]["last_sent"]
        if last_sent is not None and now - last_sent >= self.timeout:
            self.stats.timeouts += 1
            self._send_packet(states[seq_num])

def _ack_checksum_is_valid(self, ack_packet):
    expected = calculate_checksum(
        ack_packet.seq_num, ack_packet.ack_num, ack_packet.payload)
    return ack_packet.checksum == expected
```

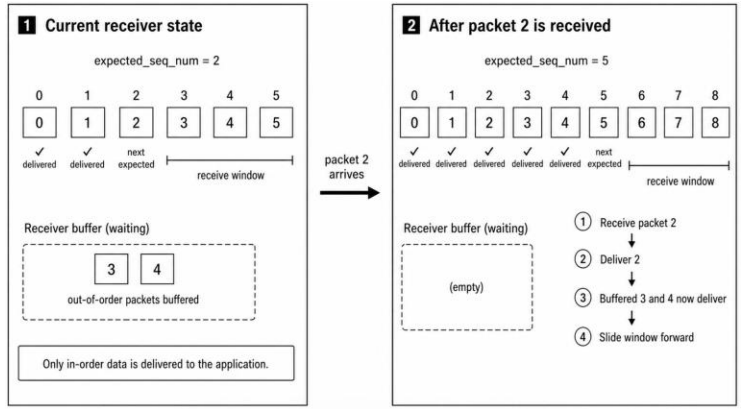
RECEIVER PACKET HANDLING

- Receive UDP packet.
- Parse packet fields.
- Recalculate checksum.
- Drop packet if checksum is invalid.
- Check if sequence number is old, inside window, or outside window.
- Send ACK for valid in-window packets.
- Resend ACK for recently received old packets.



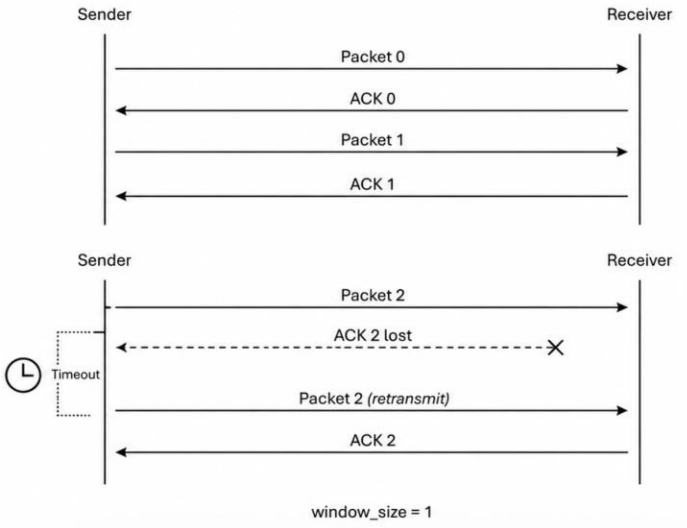
RECEIVER WINDOW + DELIVERY

- Valid packets are stored in the receiver buffer.
- Receiver tracks expected_seq_num.
- Packets are delivered only in order.
- Out-of-order packets wait in the buffer.
- When the expected packet arrives, receiver delivers it.
- Window slides forward as consecutive packets are delivered.



STOP AND WAIT

- The sender transmits one packet and starts a timer.
- The receiver validates the packet and sends back an ACK.
- If the ACK arrives before the timer expires, the sender moves on to the next packet.
- If the timer expires first, the sender retransmits the same packet and restarts the timer.
- The sender never has more than one packet in flight at a time.



SLIDING WINDOW

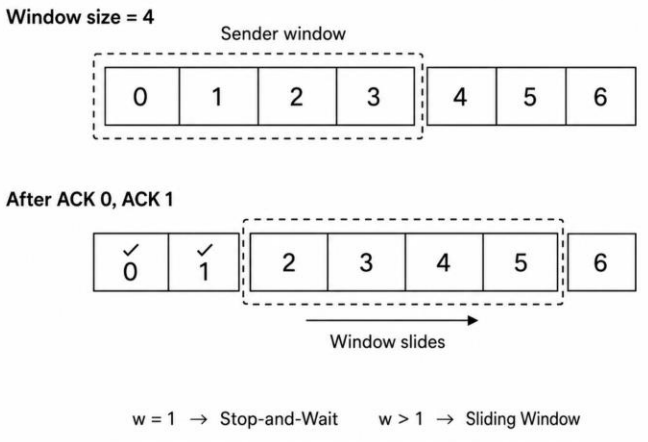
The sender can transmit any packet inside the current window without waiting for earlier ACKs.

Incoming ACKs identify which packets were received successfully.

ACKed packets are removed from the set of unacknowledged packets in flight.

The window slides forward as the earliest unacked packets are confirmed.

Any packet whose timer expires before its ACK arrives is retransmitted individually.



TESTING SETUP

- Baseline test: no artificial loss, delay, or corruption; confirms the normal sender/receiver path works.
- Delay test: adds 100 ms network delay; checks that timeout settings do not cause unnecessary retransmits.
- Loss test: drops 5% of packets; verifies the sender times out and retransmits missing data.
- Corruption tests: corrupts 2% and 10% of traffic; verifies checksum mismatches are ignored and recovered by retransmission.
- Window test: runs with window_size = 4; checks that multiple packets can be in flight while ACKs slide the window forward.
- Rigorous test: combines 75 ms delay, 10 ms jitter, 5% packet loss, 3% corruption, and 1% packet duplication with window_size = 4 to stress the protocol under harsher network conditions.
- Extreme rigorous test: combines 100 ms delay, 20 ms jitter, 10% packet loss, 5% corruption, 2% packet duplication, and 5% reordering with window_size = 4; shows the protocol's limit when the sender exceeds the retransmission cap.
- Each run starts a fresh receiver and saves matching sender/receiver logs under linux_test_results/.
- Metrics collected: original packets, packets sent, retransmissions, timeouts, and ACKs received.

TEST RESULTS

The extreme rigorous test failed because one packet did not receive a valid ACK after 50 retransmissions. Since the sliding window could not move past that missing sequence number, the sender stopped at the configured retry limit instead of hanging forever.

Scenario	Result	Original	Sent	Retrans	Timeouts	ACKs
Baseline	Pass	10	10	0	0	10
100 ms delay	Pass	10	10	0	0	10
5% packet loss	Pass	10	12	2	2	10
2% corruption	Pass	11	11	0	0	11
10% corruption	Pass	11	13	2	2	11
Window size 4	Pass	11	11	0	0	11
Combined harsh test	Pass	36	92	56	56	36
Extreme rigorous test	Fail	152	93	78	79	14

Key signal: loss/corruption increased packets sent, timeouts, and retransmissions; the combined harsh test passed, while the extreme rigorous test exceeded the retransmission limit.

DEMO

```
joe@joe:~/SpringProject$ cd ~/SpringProject/rdt-udp
python3 scripts/run_proxy_tests.py --results-dir /tmp/proxy_demo --tests all_tests
all_tests: PASS
joe@joe:~/SpringProject/rdt-udp$ grep -n "\[SENDER\] sending seq" /tmp/proxy_demo/all_tests_sender.log | head -20
5:[SENDER] sending seq=0, attempt=1
6:[SENDER] sending seq=1, attempt=1
7:[SENDER] sending seq=2, attempt=1
8:[SENDER] sending seq=3, attempt=1
13:[SENDER] sending seq=4, attempt=1
18:[SENDER] sending seq=5, attempt=1
19:[SENDER] sending seq=6, attempt=1
20:[SENDER] sending seq=7, attempt=1
23:[SENDER] sending seq=8, attempt=1
28:[SENDER] sending seq=9, attempt=1
29:[SENDER] sending seq=10, attempt=1
38:[SENDER] sending seq=7, attempt=2
41:[SENDER] sending seq=11, attempt=1
42:[SENDER] sending seq=12, attempt=1
43:[SENDER] sending seq=13, attempt=1
44:[SENDER] sending seq=14, attempt=1
53:[SENDER] sending seq=15, attempt=1
54:[SENDER] sending seq=16, attempt=1
59:[SENDER] sending seq=15, attempt=2
joe@joe:~/SpringProject/rdt-udp$ grep -n "printing the extracted packet" /tmp/proxy_demo/all_tests_receiver.log | head -30
4:printing the extracted packet: Packet(seq_num=0, ack_num=0, payload=Reliable, checksum=14)
8:printing the extracted packet: Packet(seq_num=2, ack_num=0, payload=-tests p, checksum=61)
11:printing the extracted packet: Packet(seq_num=3, ack_num=0, payload=roxy run, checksum=135)
14:printing the extracted packet: Packet(seq_num=1, ack_num=0, payload= UDP all, checksum=177)
20:printing the extracted packet: Packet(seq_num=4, ack_num=0, payload= with de, checksum=239)
24:printing the extracted packet: Packet(seq_num=7, ack_num=0, payload=ket loss, checksum=19)
27:printing the extracted packet: Packet(seq_num=5, ack_num=0, payload=lay, jit, checksum=190)
31:printing the extracted packet: Packet(seq_num=6, ack_num=0, payload=ter, pac, checksum=17)
36:printing the extracted packet: Packet(seq_num=8, ack_num=0, payload=, corrup, checksum=247)
40:printing the extracted packet: Packet(seq_num=10, ack_num=0, payload=plicatio, checksum=171)
43:printing the extracted packet: Packet(seq_num=9, ack_num=0, payload=tion, du, checksum=93)
48:printing the extracted packet: Packet(seq_num=7, ack_num=0, payload=ket loss, checksum=19)
51:printing the extracted packet: Packet(seq_num=13, ack_num=0, payload=nd a sli, checksum=38)
54:printing the extracted packet: Packet(seq_num=14, ack_num=0, payload=ding sen, checksum=226)
57:printing the extracted packet: Packet(seq_num=12, ack_num=0, payload=ering, a, checksum=193)
60:printing the extracted packet: Packet(seq_num=11, ack_num=0, payload=n, reord, checksum=20)
67:printing the extracted packet: Packet(seq_num=15, ack_num=0, payload=der wind, checksum=17)
70:printing the extracted packet: Packet(seq_num=16, ack_num=0, payload=ow., checksum=21)
73:printing the extracted packet: Packet(seq_num=15, ack_num=0, payload=der wind, checksum=16)
joe@joe:~/SpringProject/rdt-udp$ grep -n "RESULT_STATUS" /tmp/proxy_demo/all_tests_sender.log
63:RESULT_STATUS=0
joe@joe:~/SpringProject/rdt-udp$
```

This test runs our sender and receiver through a proxy that simulates bad network conditions: delay, loss, corruption, duplication, and reordering.

At the top, the sender log shows the sender mostly sending packets in sequence order: 0, 1, 2, 3, and so on. That is expected because the sender's job is to send the data stream in order.

In the receiver log, the packets arrive out of order: 0, then 2, then 3, then 1, then 4, then 7, 5, 6. That means the proxy is successfully simulating network reordering.

The important part is that the receiver can handle that. It buffers packets that arrive early, waits for the missing sequence number, and then delivers everything to the upper layer in the correct order.

The final `RESULT_STATUS=0` means the sender finished successfully, so the reliable transfer worked even with those network impairments.

DEMO

CONCLUSION

- UDP by itself gives no reliability guarantees.
- Our project adds reliability using core transport-layer mechanisms.
- Checksums detect corrupted packets.
- Sequence numbers preserve ordering and identify duplicates.
- ACKs confirm successful delivery.
- Timers and retransmission recover from packet or ACK loss.
- Sliding window improves performance by allowing multiple packets in flight.

FUTURE WORK

- Replace the temporary text packet format with a binary-safe format.
- Support arbitrary file payloads without separator limitations.
- Add an explicit end-of-message or connection teardown packet.
- Write received payloads to a file instead of only printing delivery.
- Add automated tests for loss, corruption, duplicate packets, and reordering.
- Add unit tests and CI-style regression tests for protocol edge cases.
- Collect more performance data across window sizes and loss rates.

SOURCES

- Course material: CS5470 Computer Networks reliable data transfer concepts.
- Kurose and Ross: Computer Networking, transport-layer reliability.
- Python documentation: socket module for UDP sockets.
- Linux tc netem documentation for delay, loss, and jitter emulation.
- Project repository: sender.py, receiver.py, packet.py, utils.py, scripts/.